PID regulation when the actuator saturates

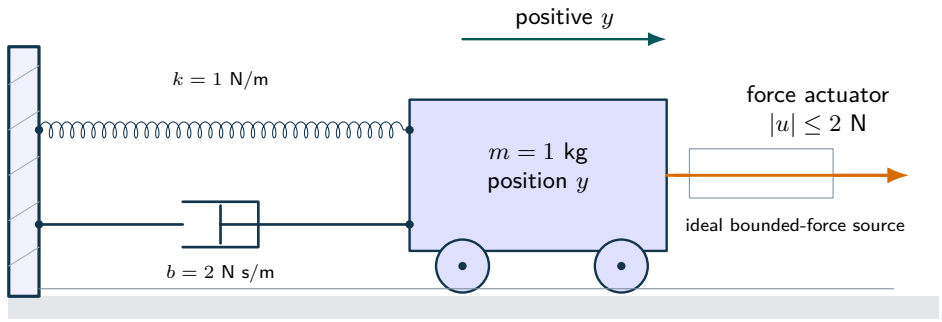
Original learning collection – November 2026

Evidence note. Original teaching scenario. Numerical parameters are deliberately chosen for learning. All generated data and curves are synthetic; no physical measurement is asserted.

1 Design brief

A normalized position plant obeys $\ddot{x} + 2\dot{x} + x = u$. Match ideal PID gains to poles $-2, -3, -4$. Implement derivative action on measured position using filter time constant $\tau_f = 0.03\,\mathrm{s}$ and constrain $u \in [-2, 2]$. Command $r = 3$ for six seconds and then $r = 0.2$. Compare ordinary integration with back-calculation gain $k_{aw} = 8\,\mathrm{s}^{-1}$ and report recovery error over the final six seconds.

2 Physical system



Illustrative force-driven stage; no contact, friction or hard stops

Force and displacement

u: applied force [N].
y: displacement [m].
Requested and applied inputs differ when saturation is active.

Dynamics and bound

$\ddot{y} + 2\dot{y} + y = u$.
$-2 \le u \le 2$ N.
The drawn source applies force to the moving mass.

Controller implication

Anti-windup must use applied-minus-requested effort. Wheels depict ideal guidance; rolling friction and hard stops are omitted.

A mass, spring, damper and bounded force actuator physically realize the normalized second-order plant. Force saturation is included; contact, dry friction and travel stops are not.

3 Ideal pole matching

For an ideal PID $K_p + K_i/s + K_d s$, the closed-loop characteristic polynomial is $s^3 + (2+K_d)s^2 + (1+K_p)s + K_i$. Matching $(s+2)(s+3)(s+4) = s^3 + 9s^2 + 26s + 24$ gives

$$K_p = 25, \qquad K_i = 24, \qquad K_d = 7.$$

These desired poles describe an ideal linear controller, not the filtered saturated implementation.

4 Realizable controller states

Let z be a low-pass filtered position, $\tau_f \dot{z} = x - z$, and let I denote the integral contribution in actuator units. Define

$$e = r - x, \quad u_* = K_p e + I - K_d(x - z)/\tau_f, \quad u = \mathrm{clip}(u_*, -2, 2).$$

Derivative action is applied to the measurement, avoiding the ideal derivative impulse from a reference step. The unsaturated characteristic polynomial becomes

$$\tau_f s^4 + (1 + 2\tau_f)s^3 + [2 + \tau_f(1 + K_p) + K_d]s^2 + [1 + K_p + K_i\tau_f]s + K_i.$$

Its roots must be checked separately.

5 Windup and back-calculation

Ordinary integration uses $\dot{I} = K_i e$. Back-calculation uses $\dot{I} = K_i e + 8(u - u_*)$. The second term reduces the mismatch between requested and realizable actuator effort. It vanishes when unsaturated. The initial command 3 is infeasible because the plant's static gain is one and the actuator limit is 2. The experiment intentionally accumulates windup, then commands a feasible target. Integrate the two time intervals separately so the reference discontinuity is not smoothed by the solver.

6 Assessment

Compare the integral state, output, actuator and integral absolute error after the command reduction. Back-calculation improves this particular recovery; its gain is not a universal optimum and noisy measurements or other saturation limits can change the result.

7 What to learn from the experiment

An infeasible command cannot be repaired by controller gain. Anti-windup improves recovery after saturation rather than increasing physical actuator capacity.

8 System sketch

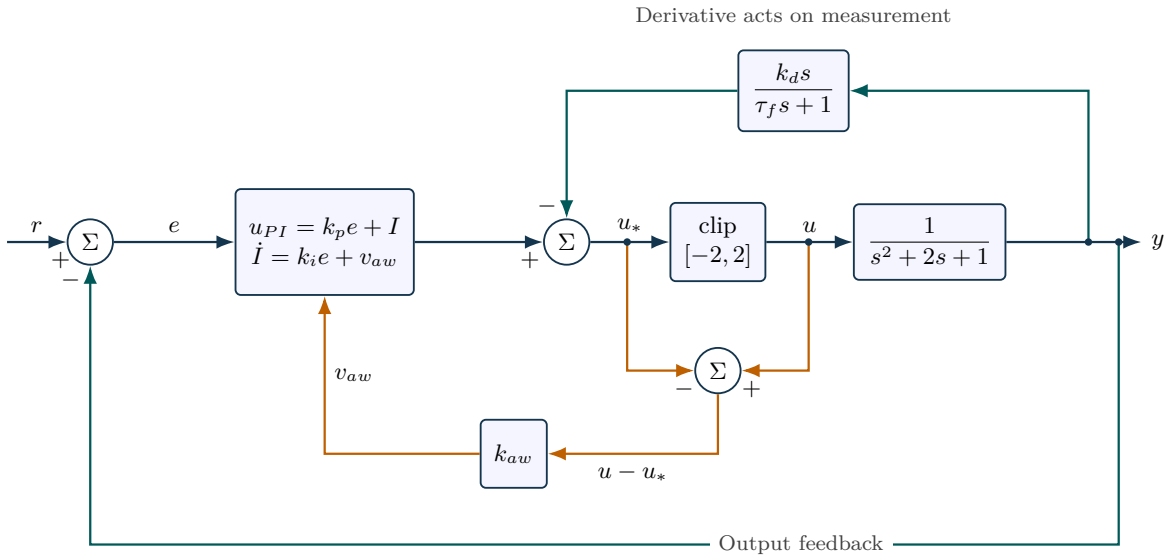
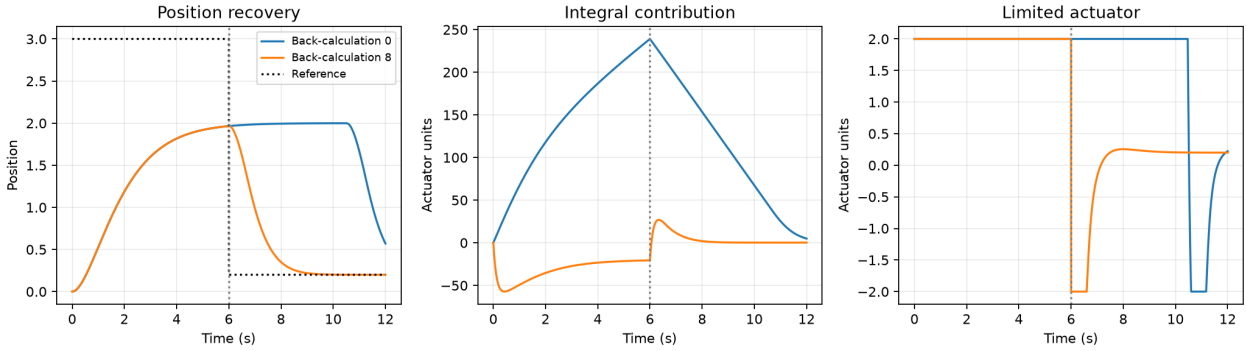


Figure 1. Filtered derivative acts on measurement. Back-calculation sends the actual-minus-requested actuator effort into the integral-state equation.

9 Executed computational evidence

```
Back-calculation gain 0: recovery IAE=9.756216, final x=0.570287
Back-calculation gain 8: recovery IAE=1.841207, final x=0.199941
```



10 Method references

The equations use standard balance laws and feedback-control methods. Further explanation: https://apmonitor.com/pdc/ and https://fbsbook.org/. Implementation documentation: https://python-control.readthedocs.io/en/0.10.2/. See the repository reference catalog for individual topics and videos.